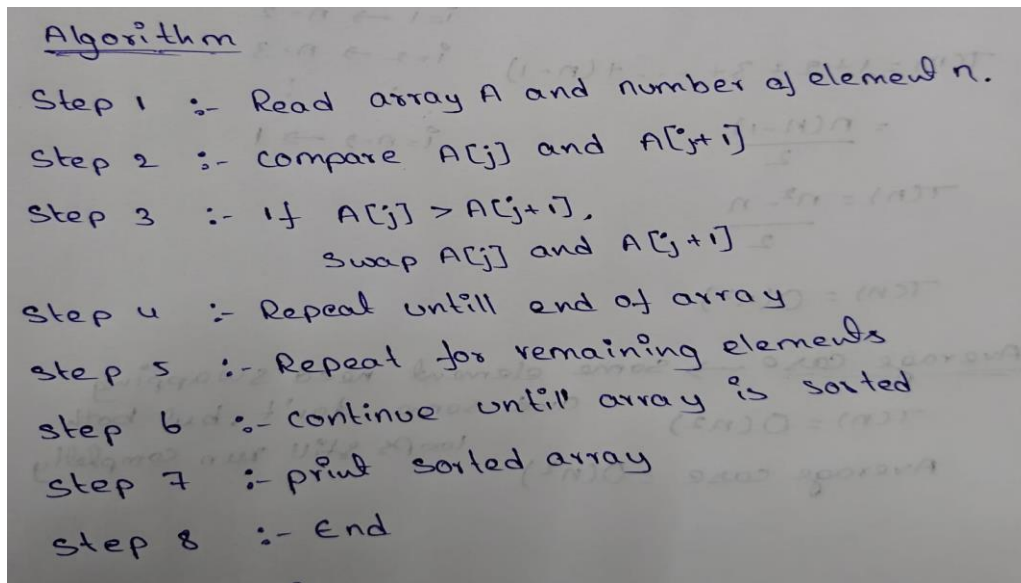


Practical - 1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1)Bubble Sort

Algorithm:



Program:

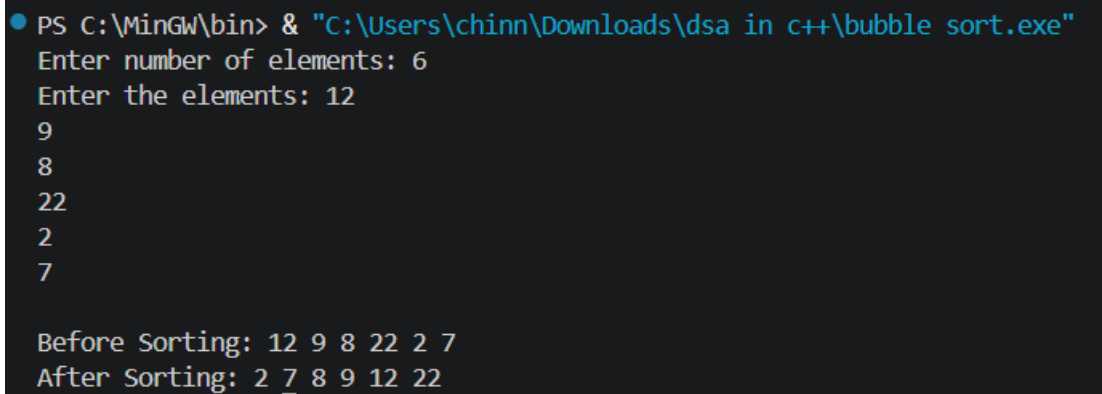
```
#include <iostream>
using namespace std;
class BubbleSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printArray(){
        for (int i = 0; i < n; i++){
            cout << arr[i] << " ";
        }
    }
};
```

Name: Chinni Thabitha
Enrollment Number:92460118038

```
        cout << endl;
    }
    void logic(){
        int temp;
        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - i - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
};
int main()
{
    BubbleSort bs;
    bs.arrayInput();
    cout << "\nBefore Sorting: ";
    bs.printArray();
    bs.logic();
    cout << "After Sorting: ";
    bs.printArray();
    return 0;
}
```

Output:



```
● PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\bubble sort.exe"
Enter number of elements: 6
Enter the elements: 12
9
8
22
2
7

Before Sorting: 12 9 8 22 2 7
After Sorting: 2 7 8 9 12 22
```

Time Complexity:

Time complexity

for ($i=0; i < n-1; i++$) { $\rightarrow O(n)$
 for ($j=0; j < n-1; j++$) { $\rightarrow O(n)$
 if ($A[j] > A[j+1]$) { $\rightarrow (1)$
 swap($A[j], A[j+1]$); $\rightarrow (1)$
 }
}

$T(n) = n \times n = n^2 = O(n^2)$
 Best case = $O(n^2) \rightarrow$ no swap required

worst case $\rightarrow$ many swaps are required

$(n-1) + (n-2) + (n-3) + \dots + 1$
 $T(n) = 1 + 2 + 3 + \dots + (n-1)$
 $= \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

Average case $\rightarrow$ some element need swapping and some don't, but both loops still run completely
 $T(n) = O(n^2)$
 Average case = $O(n^2)$

2) Selection sort:

Algorithm:

Step 1 :- Read array A and number of elements n
 Step 2 :- set $min = i$
 Step 3 :- Compare $A[min]$ with $A[j]$
 Step 4 :- If $A[j] < A[min]$,
 set $min = j$
 Step 5 :- Repeat steps 3 and step 4
 until the end of array
 Step 6 :- swap $A[i]$ and $A[min]$
 Step 7 :- Repeat the above steps
 for remaining elements
 Step 8 :- print sorted array
 Step 9 :- end

Program:

```
#include <iostream>

using namespace std;

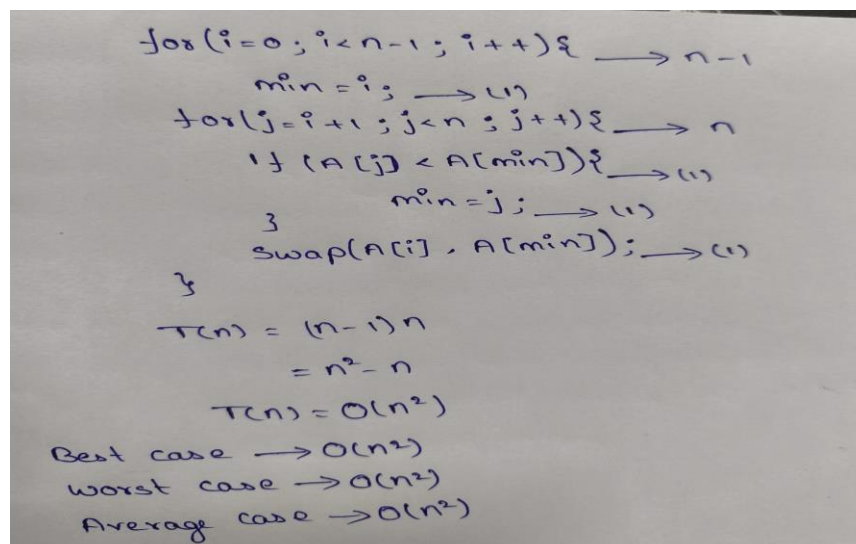
class SelectionSort{
public:
    int arr[100], n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void logic(){
        int i, j, min, temp;
        for (i = 0; i < n - 1; i++){
            min = i;
            for (j = i + 1; j < n; j++){
                if (arr[j] < arr[min]){
                    min = j;
                }
            }
            Swap(arr[i],arr[min]);
        }
    }
    void printArray() {
        for (int i = 0; i < n; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}
```

```
};  
  
int main()  
{  
    SelectionSort ss;  
    ss.arrayInput();  
    cout << "Before Sorting: ";  
    ss.printArray();  
    ss.logic();  
    cout << "After Sorting: ";  
    ss.printArray();  
    return 0;  
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\selectionsort.exe"  
Enter number of elements: 6  
Enter the elements: 12  
9  
2  
8  
31  
1  
Before Sorting: 12 9 2 8 31 1  
After Sorting: 1 2 8 9 12 31
```

Time complexity:

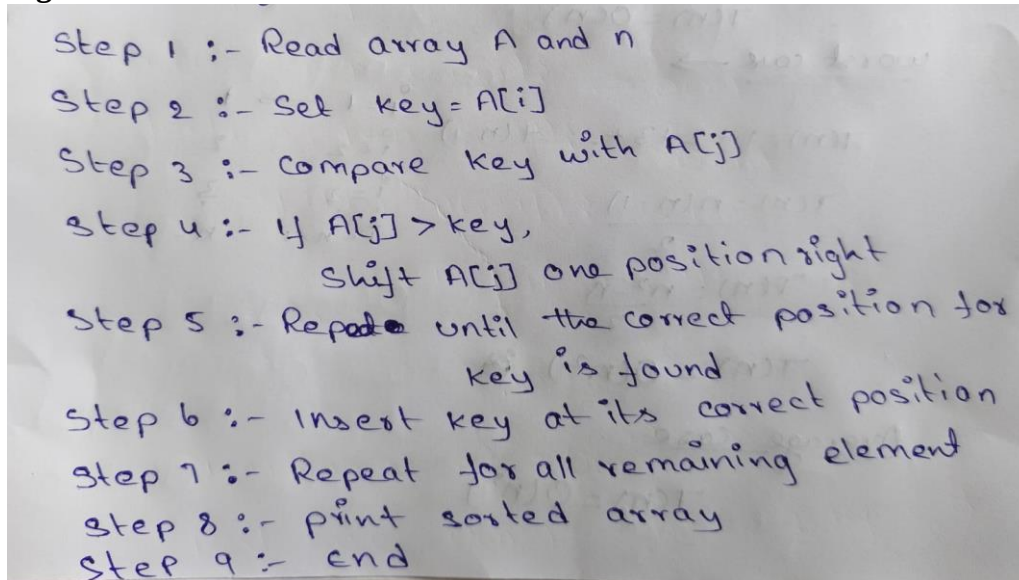


for (i=0; i<n-1; i++) { $\rightarrow n-1$
 min = i; $\rightarrow (1)$
 for (j=i+1; j<n; j++) { $\rightarrow n$
 if (A[j] < A[min]) { $\rightarrow (1)$
 min = j; $\rightarrow (1)$
 }
 swap(A[i], A[min]); $\rightarrow (1)$
 }
}

 $T(n) = (n-1)n$
 $= n^2 - n$
 $T(n) = O(n^2)$
Best case $\rightarrow O(n^2)$
Worst case $\rightarrow O(n^2)$
Average case $\rightarrow O(n^2)$

3) Insertion sort:

Algorithm:



Step 1 :- Read array A and n
Step 2 :- Set key = A[i]
Step 3 :- Compare key with A[j]
Step 4 :- If A[j] > key,
 Shift A[j] one position right
Step 5 :- Repeat until the correct position for
 key is found
Step 6 :- Insert key at its correct position
Step 7 :- Repeat for all remaining element
Step 8 :- print sorted array
Step 9 :- end

Program:

```
#include <iostream>
using namespace std;
class InsertionSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void logic()
    {
        int i, j, key;

        for (i = 1; i < n; i++)
        {
            key = arr[i];
            j = i - 1;

            while (j >= 0 && arr[j] > key)
            {
```

```
        arr[j + 1] = arr[j];
        j--;
    }

    arr[j + 1] = key;
}
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    InsertionSort is;

    is.arrayInput();

    cout << "Before Sorting: ";
    is.printArray();

    is.logic();

    cout << "After Sorting: ";
    is.printArray();

    return 0;
}
```

Output:



```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\insertionsort.exe"
Enter number of elements: 6
Enter the elements: 12
9
3
32
4
15
Before Sorting: 12 9 3 32 4 15
After Sorting: 3 4 9 12 15 32
```


Time Complexity:

```

for (i = 1; i < n; i++) { → n-1
    key = A[i]; → 1
    j = i - 1; → 1
    while (j >= 0 && A[j] > key) {
        A[j+1] = A[j]; → 1
        j--; → 1
    }
    A[j+1] = key; → 1
}

```

$T(n) = (n-1) + 1 + 1 + 1$
 $T(n) = n + 2$
 $T(n) = O(n) \rightarrow$ in Best case

Ex:- $A = [1, 2, 3, 4, 5] \rightarrow$ while loop does not shift any element
 $T(n) = O(n)$

worst case →

$T(n) = 1 + 2 + 3 + \dots + (n-1)$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2)$

Average case
 $T(n) = O(n^2)$

4) Merge Sort:

Algorithm:

```

Step 1 :- Read array A and n
Step 2 :- Divide the array into two halves
Step 3 :- Recursively sort the left half.
Step 4 :- Recursively sort the right half
Step 5 :- merge both sorted halves.
Step 6 :- Repeat until the whole array is sorted.
Step 7 :- print the sorted array
Step 8 :- end.

```


Program:

```
#include <iostream>
using namespace std;
class MergeSort{
public:
    int arr[100],n;
    void arrayInput(){
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++){
            cin >> arr[i];
        }
    }
    void merge(int low, int mid, int high){
        int temp[100];
        int i = low;
        int j = mid + 1;
        int k = low;
        while (i <= mid && j <= high){
            if (arr[i] < arr[j]){
                temp[k] = arr[i];
                i++;
            }
            else{
                temp[k] = arr[j];
                j++;
            }
            k++;
        }
        while (i <= mid)
        {
            temp[k] = arr[i];
            i++;
            k++;
        }
        while (j <= high)
        {
            temp[k] = arr[j];
            j++;
            k++;
        }
        for (i = low; i <= high; i++)
        {
            arr[i] = temp[i];
        }
    }
    void logic(int low, int high)
    {
```

```
if (low < high)
{
    int mid = (low + high) / 2;
    logic(low, mid);
    logic(mid + 1, high);
    merge(low, mid, high);
}
}
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
};

int main()
{
    MergeSort ms;

    ms.arrayInput();

    cout << "Before Sorting: ";
    ms.printArray();

    ms.logic(0, ms.n - 1);

    cout << "After Sorting: ";
    ms.printArray();

    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\mergesort.exe"
Enter number of elements: 6
Enter the elements: 15
9
6
2
22
7
Before Sorting: 15 9 6 2 22 7
After Sorting: 2 6 7 9 15 22
```



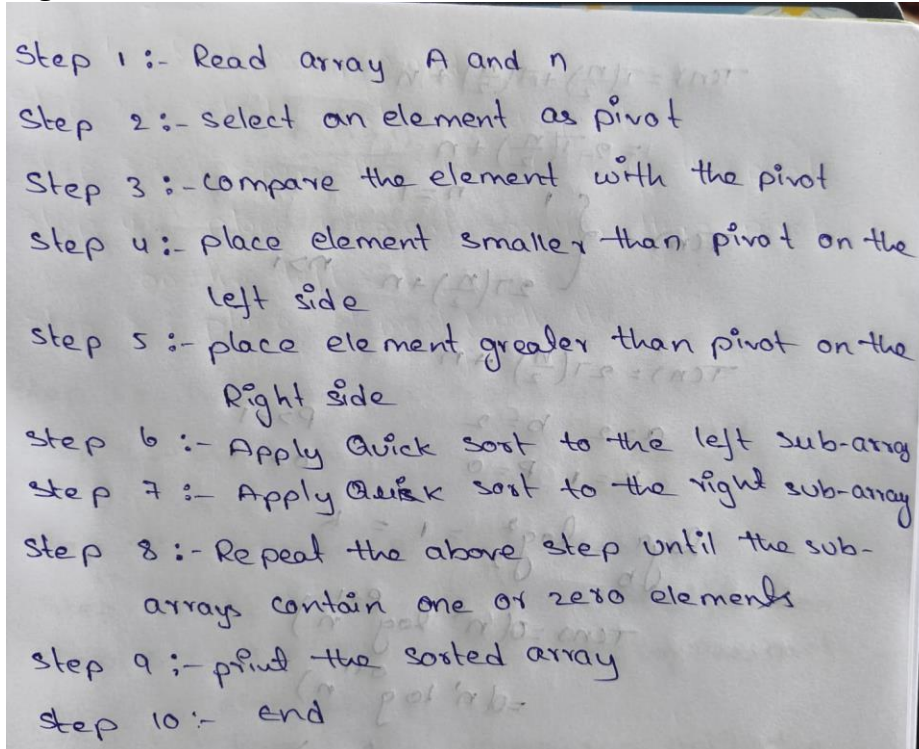
Time Complexity:

$$\begin{aligned}
 &\text{MergeSort}(A, \text{low}, \text{high}) \{ \rightarrow T(n) \\
 &\quad \text{if } (\text{low} < \text{high}) \{ \rightarrow 1 \\
 &\quad \quad \text{mid} = (\text{low} + \text{high}) / 2; \rightarrow 1 \\
 &\quad \quad \text{mergeSort}(A, \text{low}, \text{mid}); \rightarrow T\left(\frac{n}{2}\right) \\
 &\quad \quad \text{mergeSort}(A, \text{mid} + 1, \text{high}); \rightarrow T\left(\frac{n}{2}\right) \\
 &\quad \quad \text{merge}(A, \text{low}, \text{mid}, \text{high}); \rightarrow n \\
 &\quad \} \\
 &\} \\
 &T(n) = \begin{cases} 1 & n = 1 \\ 2T\left(\frac{n}{2}\right) + n & n > 1 \end{cases} \\
 &T(n) = 2T\left(\frac{n}{2}\right) + n
 \end{aligned}$$

$$\begin{aligned}
 T(n) &= 2T\left(\frac{n}{2}\right) + n \\
 T(n) &= 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\right] + n \\
 &= 2^2 T\left(\frac{n}{4}\right) + n + n \\
 &= 2^2 T\left(\frac{n}{4}\right) + 2n \\
 T(n) &= 2^2 \left[2T\left(\frac{n}{8}\right) + \frac{n}{4}\right] + 2n \\
 &= 2^3 T\left(\frac{n}{8}\right) + n + 2n \\
 &= 2^3 T\left(\frac{n}{8}\right) + 3n \\
 T(n) &= 2^3 \left[2T\left(\frac{n}{16}\right) + \frac{n}{8}\right] + 3n \\
 &= 2^4 T\left(\frac{n}{16}\right) + n + 3n \\
 &= 2^4 T\left(\frac{n}{16}\right) + 4n \\
 T(n) &= 2^k T\left(\frac{n}{2^k}\right) + kn \Rightarrow \\
 n &= 1 \\
 \frac{n}{2^k} &= 1 \\
 n &= 2^k \\
 \text{Apply log on both side} \\
 \log n &= \log 2^k \\
 \log n &= k \log 2 \\
 \frac{\log n}{\log 2} &= k \\
 k &= \log_2 n \\
 T(n) &= n T\left(\frac{n}{n}\right) + n \log_2 n \\
 &= n(1) + n \log_2 n \\
 &= n + n \log_2 n \\
 T(n) &= \Theta(n \log n) \\
 \text{Best case} &= O(n \log n) \\
 \text{worst case} &= O(n \log n) \\
 \text{Average case} &= O(n \log n)
 \end{aligned}$$

5)Quick Sort:

Algorithm:



Step 1 :- Read array A and n
Step 2 :- select an element as pivot
Step 3 :- compare the element with the pivot
Step 4 :- place element smaller than pivot on the left side
Step 5 :- place element greater than pivot on the right side
Step 6 :- Apply Quick sort to the left sub-array
Step 7 :- Apply Quick sort to the right sub-array
Step 8 :- Repeat the above step until the sub-arrays contain one or zero elements
Step 9 :- print the sorted array
Step 10 :- end

Program:

```
#include <iostream>
using namespace std;
class QuickSort
{
public:
    int arr[100], n;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }
    }
};
```

Name: Chinni Thabitha

Enrollment Number:92460118038

```
    cout << endl;
}

int partition(int arr[], int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (arr[start] <= pivot && start < ub)
        {
            start++;
        }

        while (arr[end] > pivot)
        {
            end--;
        }

        if (start < end)
        {
            swap(arr[start], arr[end]);
        }
    }

    swap(arr[lb], arr[end]);

    return end;
}

void quicksort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int location = partition(arr, lb, ub);

        quicksort(arr, lb, location - 1);
        quicksort(arr, location + 1, ub);
    }
}

void sort()
{
    quicksort(arr, 0, n - 1);
}

};

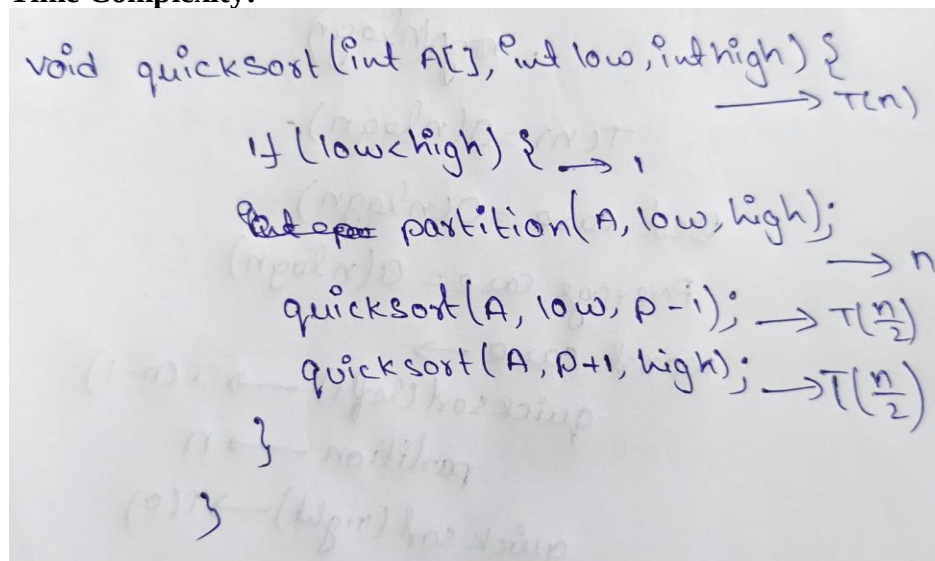
int main()
```

```
{  
    QuickSort qs;  
  
    qs.arrayInput();  
  
    cout << "\nBefore Sorting: ";  
    qs.printArray();  
  
    qs.sort();  
  
    cout << "After Sorting: ";  
    qs.printArray();  
  
    return 0;  
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\quicksort.exe"  
Enter number of elements: 6  
Enter the elements: 15  
9  
3  
4  
23  
2  
  
Before Sorting: 15 9 3 4 23 2  
After Sorting: 2 3 4 9 15 23
```

Time Complexity:



```
void quicksort(int A[], int low, int high) {  
    //  $\rightarrow T(n)$   
    if (low < high) {  $\rightarrow 1$   
        But op partition(A, low, high);  $\rightarrow n$   
        quicksort(A, low, p-1);  $\rightarrow T(\frac{n}{2})$   
        quicksort(A, p+1, high);  $\rightarrow T(\frac{n}{2})$   
    }  
}
```




$$\begin{aligned}
 T(n) &= T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n \\
 &= 2T\left(\frac{n}{2}\right) + n \\
 T(n) &= \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + n & n>1 \end{cases} \\
 T(n) &= 2T\left(\frac{n}{2}\right) + n \\
 a=2 \quad b=2 \quad p>-1 \\
 k=1 \quad p=0 \quad o>-1 \\
 \log_a b &= \log_2 2 = 1 = k \\
 T(n) &= O(n^k \log^{p+1} n) \\
 &= O(n^1 \log^{0+1} n) \\
 &= O(n \log n) \\
 T(n) &= O(n \log n) \\
 \text{Best case} &= O(n \log n) \\
 \text{Average case} &= O(n \log n) \\
 \text{worst case} &\rightarrow \\
 \text{quicksort(left)} &\rightarrow T(n-1) \\
 \text{partition} &\rightarrow n \\
 \text{quicksort(right)} &\rightarrow T(0) \\
 T(n) &= T(n-1) + n \\
 T(n) &= n + (n-1) + (n-2) + \dots + 1 \\
 &= \frac{n(n+1)}{2} \\
 &= \frac{n^2 + n}{2} \\
 T(n) &= O(n^2)
 \end{aligned}$$

Conclusion:

The implementation and time analysis of Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort helps us understand how different sorting algorithms perform for different input sizes. By analyzing their best, average, and worst cases, we can choose the most suitable algorithm for a particular problem. Merge Sort and Quick Sort are generally faster for large datasets, while Bubble, Selection, and Insertion Sort are simpler and useful for small datasets.